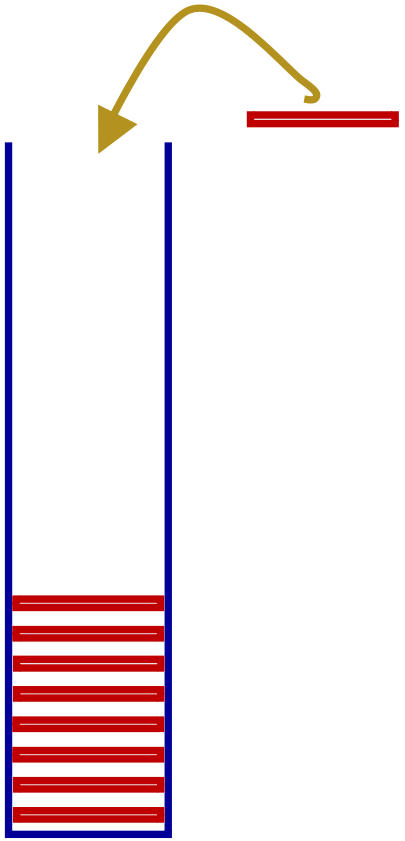
# ESTRUTURAS DE DADOS BÁSICAS PARTE 1/3

## PILHA E FILA EM ALOCAÇÃO SEQUENCIAL

# Pilha



LIFO (*last-in-first-out*)

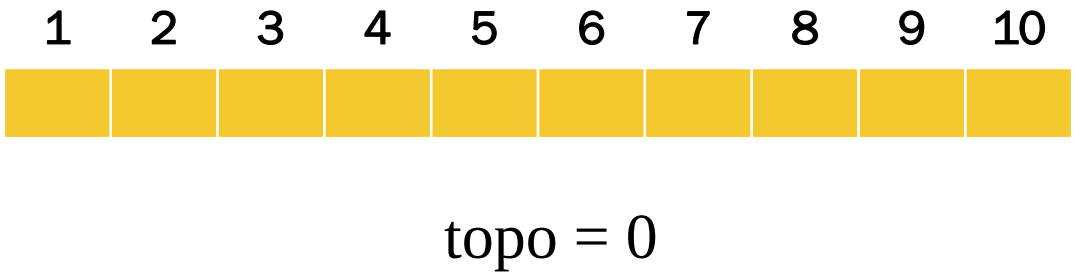
Último a entrar, primeiro a sair

Métodos:
Inicia_pilha
Pilha_vazia
Empilha(x,P)
Desempilha(P)

Complexidade $\Theta(1)$

# Pilha – Alocação Sequencial

```
Inicia_Pilha
   topo ← 0
```




```
Pilha_vazia
  se topo = 0
      retorne (verdadeiro)
  senão retorne (falso)
```

# Pilha – Alocação Sequencial

```
Inicia_Pilha
   topo ← 0
```

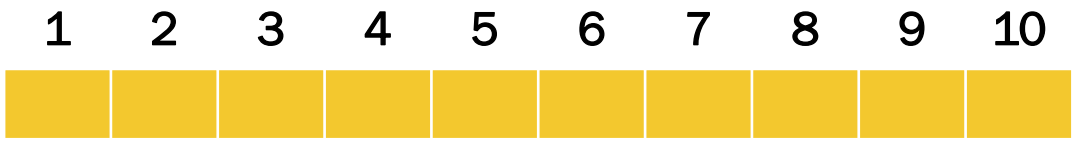
```
Pilha_vazia
  se topo = 0
      retorne (verdadeiro)
  senão retorne (falso)
```

Complexidade
Inicia_Pilha: Θ(1)
Pilha_vazia: Θ(1)

# Pilha – Alocação Sequencial

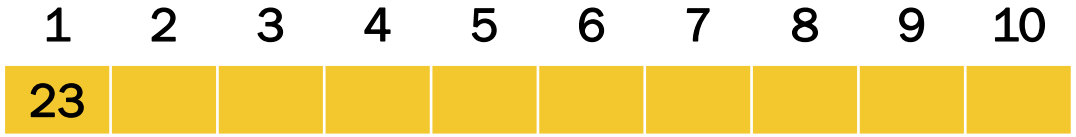
```
Empilha(x,P)
   se topo = limite
      retorna (“erro: acima do limite”)
   topo ← topo + 1
   P[topo] ← x
```

Antes




topo = 0, x= 23
limite = 10

Depois




topo = 1

# Pilha – Alocação Sequencial

```
Empilha(x,P)
   se topo = limite
      retorna (“erro: acima do limite”)
   topo ← topo + 1
   P[topo] ← x
```

Antes




topo = 5, x= 4
limite = 10

Depois




topo = 6

# Pilha – Alocação Sequencial

```
Empilha(x,P)
   se topo = limite
      retorna (“erro: acima do limite”)
   topo ← topo + 1
   P[topo] ← x
```

Qual a complexidade de melhor e pior caso do procedimento Empilha?

# Pilha – Alocação Sequencial

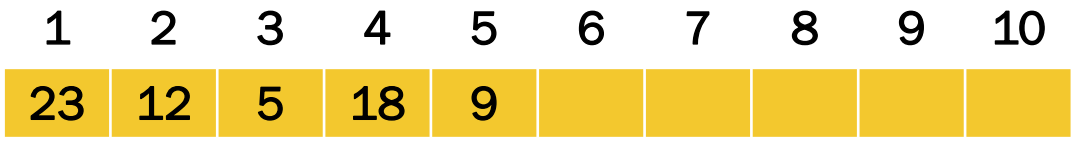
```
Empilha(x,P)
   se topo = limite
      retorna (“erro: acima do limite”)
   topo ← topo + 1
   P[topo] ← x
```

$\Theta(1)$

# Pilha – Alocação Sequencial

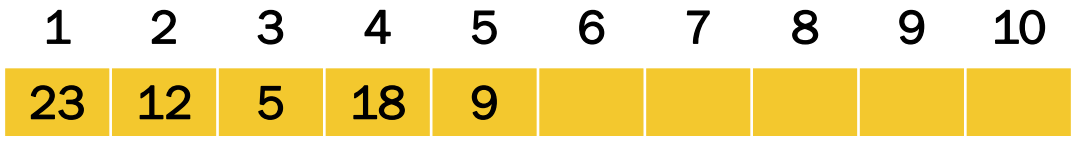
```
Desempilha(P)
  se (pilha_vazia)
    retorna (“erro: Pilha vazia”)
  topo ← topo – 1
  retorna(P[topo+1])
```

Antes




topo = 5

Depois




topo = 4

# Pilha – Alocação Sequencial

```
Desempilha(P)
  se (pilha_vazia)
    retorna (“erro: Pilha vazia”)
  topo ← topo – 1
  retorna(P[topo+1])
```

Qual a complexidade de melhor e pior caso do procedimento Desempilha?

# Pilha – Alocação Sequencial

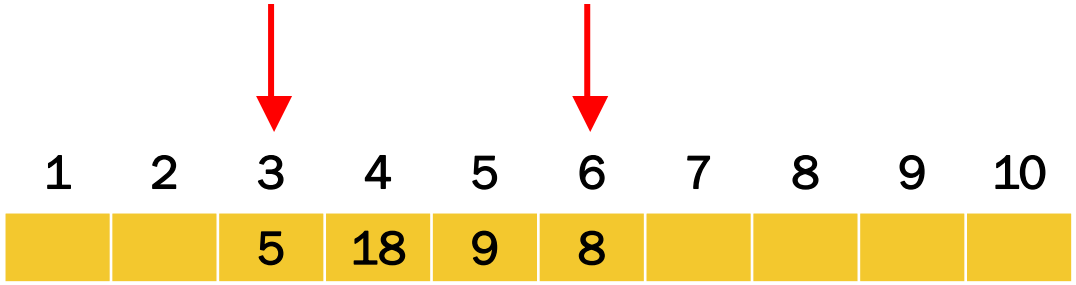
```
Desempilha(P)
  se (pilha_vazia)
    retorna (“erro: Pilha vazia”)
  topo ← topo – 1
  retorna(P[topo+1])
```

$\Theta(1)$

# Fila – Alocação Sequencial

**FIFO (*first-in-first-out*)**
**Primeiro a entrar, primeiro a sair**

**Métodos:**
**Inicia_fila**
**Fila_vazia**
**Insere(x,F)**
**Remove(F)**




início = 3
fim = 7

Fila entre posições início e fim-1

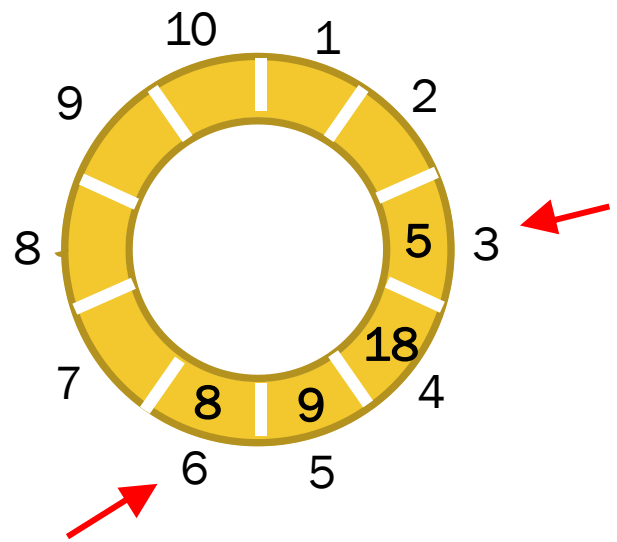
Fila circular

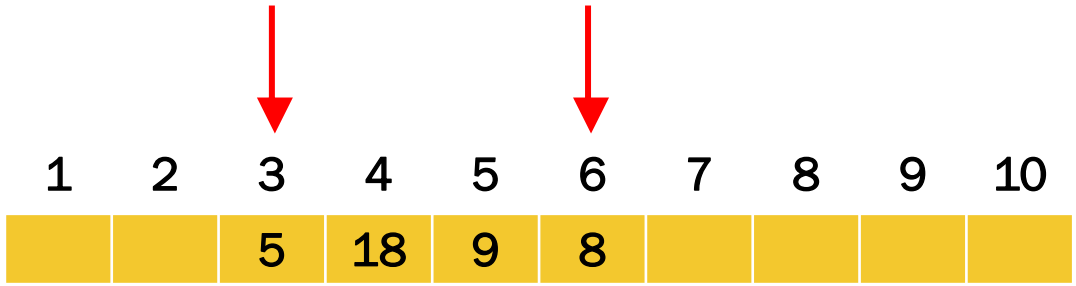
# Fila – Alocação Sequencial

**FIFO (*first-in-first-out*)**
**Primeiro a entrar, primeiro a sair**

**Métodos:**
**Inicia_fila**
**Fila_vazia**
**Insere(x,F)**
**Remove(F)**

Fila circular

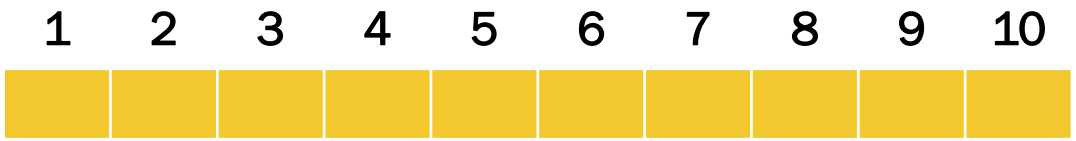



início = 3
fim = 7

# Fila – Alocação Sequencial

```
Inicia_Fila
   início ← 1
   fim ← 1
```




início = 1
fim = 1

```
Fila_vazia
  se início = fim
    retorne (verdadeiro)
  senão retorne (falso)
```

# Fila – Alocação Sequencial

```
Inicia_Fila
   início ← 1
   fim ← 1
```

```
Fila_vazia
  se início = fim
    retorne (verdadeiro)
  senão retorne (falso)
```

Complexidade

Inicia_Fila: Θ(1)

Fila_vazia: Θ(1)

# Fila – Alocação Sequencial

```
Inicia_Fila
   início ← 1
   fim ← 1
```

```
Fila_vazia
  se início = fim
    retorne (verdadeiro)
  senão retorne (falso)
```

E como identificar que a fila está cheia?

# Fila – Alocação Sequencial

```
Inicia_Fila
   início ← 1
   fim ← 1
```

início = 1 e fim = limite
ou
início = fim + 1

```
Fila_vazia
  se início = fim
    retorne (verdadeiro)
  senão retorne (falso)
```

# Fila – Alocação Sequencial

```
Insere(x,F)
   F[fim] ← x
   se (fim = limite)
      fim ← 1
   senão fim ← fim + 1
```

Antes




início = 3, x = 4
fim = 7, limite = 10

Depois




início = 3
fim = 8

**Exercício: tratar a possibilidade de *overflow***

# Fila – Alocação Sequencial

```
Insere(x,F)
   F[fim] ← x
   se (fim = limite)
      fim ← 1
   senão fim ← fim + 1
```

Complexidade
Insere: Θ(1)

# Fila – Alocação Sequencial

```
Remove(F)
  se (Fila_vazia)
     retorne("erro: fila vazia")
  x←F[início]
  se (início = limite)
       início ← 1
   senão
       início ← início + 1
   retorne(x)
```

Antes




início = 3
fim = 7

Depois




início = 4
fim = 7

# Fila – Alocação Sequencial

```
Remove(F)
  se (Fila_vazia)
     retorne("erro: fila vazia")
  x←F[início]
  se (início = limite)
       início ← 1
   senão
       início ← início + 1
   retorne(x)
```

Complexidade
Remove: Θ(1)

# Deque – Alocação Sequencial

**Fila de Dupla extremidade (*Double ended queue*)**

**Permite inserção e remoção em ambas extremidades.**

**Exercício:**

**Escreva procedimentos, em Θ(1), para inserir e remover elementos de uma deque.**